CONFLUENT KAFKA

ADMINISTRATION

Rajesh Pasham

Platform Overview

Confluent & Kafka foundations

Core Architecture

Topics · Partitions · Brokers

KRaft Mode

ZooKeeper-free metadata management

Admin & Operations

Configs · Tools · Best practices

What Is Confluent Platform?

The enterprise-grade streaming platform built by Kafka's original creators

Full-Scale Data Streaming Platform

Built by the original creators of Apache Kafka®, Confluent expands Kafka's benefits with enterprise-grade features — removing the burden of cluster management and monitoring.

80%+ of the Fortune 100 use Confluent technology

100+ pre-built Kafka connectors out of the box

1 unified platform for store, access & manage data

Why Confluent? The Business Case

From fragmented data pipelines to a single, real-time source of truth

Event-Driven Apps

Build modern apps that react to business events in real time rather than batch processes.

Universal Data Pipeline

Replace brittle point-to-point integrations with a scalable, managed streaming backbone.

Fraud Detection

Detect anomalies in financial transactions within milliseconds using stream processing.

IoT & Autonomous Cars

Ingest millions of sensor and telemetry events per second with zero data loss.

Microservices

Decouple services with async event-based communication for independent deployments.

Omnichannel Retail

Synchronize inventory, orders and customer data across all channels in real time.

01

Kafka Fundamentals

Events · Topics · Partitions · Brokers · Producers · Consumers

What Is an Event?

The fundamental unit of data in Apache Kafka

"An event is just a thing that has happened — combined with a description of what happened."

Key + Value

Kafka models every event as a key/value pair.
The key identifies the entity (e.g., customer ID).
The value holds the event payload (JSON, Avro, Protobuf).

IoT & Smart Devices

Thermostat, sensor, GPS telemetry pinging home.

Business Process Change

Invoice past due, order shipped, payment cleared.

User Interactions

Clicks, searches, form submissions, page views.

Microservice Output

Service completes a unit of work and emits a fact.

Kafka Topics

A durable, append-only log of events — the core unit of organisation

Topic: user-events

offset: 0	{ event_1: "..." }
offset: 1	{ event_2: "..." }
offset: 2	{ event_3: "..." }
offset: 3	{ event_4: "..." }
offset: 4	{ event_5: "..." }
offset: 5	{ event_6: "..." }
offset: 6	*← New writes append here*

Named Container

Each topic has a unique name. Think of it like a table in a relational database.

Append-Only Log

New events always go to the end. No inserts, no deletes, no updates.

Immutable Events

Once written, events cannot be changed. History is preserved permanently.

Configurable Retention

Retention can be seconds, years, or infinite — backed by disk for durability.

Offset-Based Access

Seek to any offset and scan forward. Not indexed — simple by design.

Partitioning: Kafka's Scalability Engine

Breaking topics into parallel logs distributed across the cluster

Partition 0

msg 1

msg 4

msg 7

No Key → Round Robin

Partition 1

msg 2

msg 5

msg 8

With Key → Hash Routed

Partition 2

msg 3

msg 6

msg 9

Ordering Guarantee

Brokers & Replication

The physical infrastructure that stores, serves, and protects your data

Broker Node

A server (physical, VM, or container) running the Kafka broker process. Hosts partitions.

Partition Management

Each broker stores a subset of all topic partitions, balancing load across the cluster.

Read/Write Handling

Brokers handle all produce and consume requests — always to/from the partition leader.

Intentionally Simple

Brokers do one job well. Simplicity enables elastic scaling and easy future evolution.

Replication

Fault Tolerance

Each partition is copied to N brokers (replication factor). Default RF=3.

Leader + Followers

One leader handles all reads/writes. N-1 followers silently replicate.

Automatic Failover

If a leader dies, a follower is instantly promoted. Zero manual intervention.

Tunable Durability

acks=all ensures a write is durable before acknowledgement to producer.

Producers & Consumers

The client applications that write to and read from Kafka topics

PRODUCERS

Client application

Code you write using the KafkaProducer API.

Connection pooling

Library manages connections to brokers automatically.

Network buffering

Batches messages for efficient network utilization.

Partitioning logic

Producer decides which partition each message lands in.

Fire and forget / acks

Tunable acknowledgement for performance vs. durability.

CONSUMERS

Client application

Code you write using the KafkaConsumer API.

Reading does NOT destroy

Messages stay in topic after read — many consumers can read.

Consumer Groups

Same group ID = partitions shared; different = full independent copy.

Elastic horizontal scale

Add instances → automatic partition rebalance. No code changes.

Offset tracking

Consumer tracks its own position. Can replay by resetting offsets.

Confluent Platform Components

Apache Kafka at the core, surrounded by enterprise-grade capabilities

Schema Registry

Centralized schema management for Avro, JSON Schema & Protobuf. Ensures producer/consumer compatibility.

REST Proxy

HTTP interface to Kafka. Enables non-JVM languages to produce and consume without native client libraries.

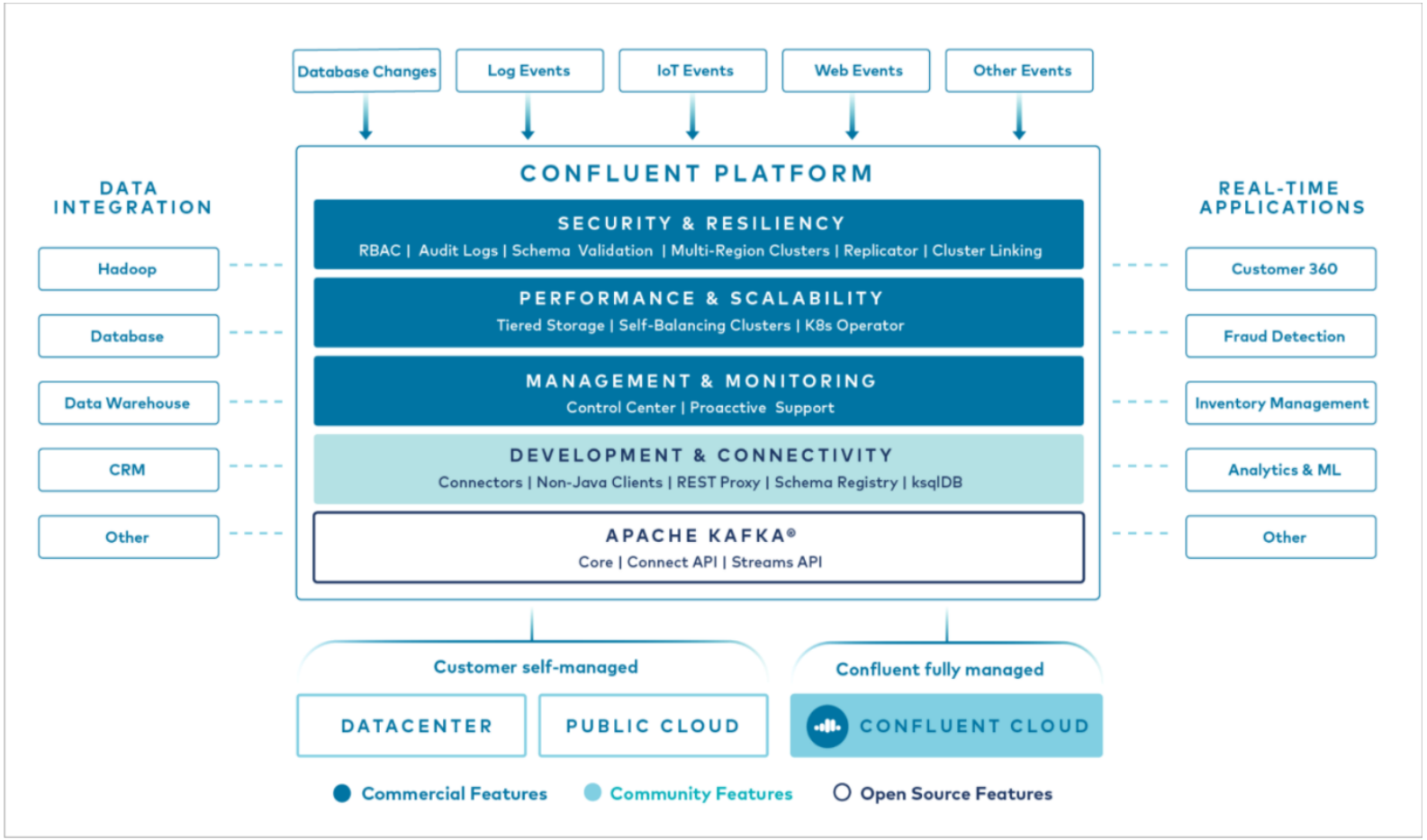
Apache Kafka

Kafka Connect

100+ pre-built connectors for databases, cloud services, and SaaS platforms. No custom code required.

ksqlDB

Stream processing with SQL syntax. Build real-time aggregations, joins and enrichments without writing code.


Database Changes
Log Events
IoT Events
Web Events
Other Events
DATA INTEGRATION
Hadoop
Database
Data Warehouse
CRM
Other
CONFLUENT PLATFORM
SECURITY & RESILIENCY
RBAC | Audit Logs | Schema Validation | Multi-Region Clusters | Replicator | Cluster Linking
PERFORMANCE & SCALABILITY
Tiered Storage | Self-Balancing Clusters | K8s Operator
MANAGEMENT & MONITORING
Control Center | Proacctive Support
DEVELOPMENT & CONNECTIVITY
Connectors | Non-Java Clients | REST Proxy | Schema Registry | ksqlDB
APACHE KAFKA®
Core | Connect API | Streams API
REAL-TIME APPLICATIONS
Customer 360
Fraud Detection
Inventory Management
Analytics & ML
Other
Customer self-managed
DATACENTER
PUBLIC CLOUD
Confluent fully managed
CONFLUENT CLOUD
Commercial Features
Community Features
Open Source Features

02

Cluster Architecture

Brokers · ZooKeeper · KRaft · Coordination & Failover

Kafka Cluster Architecture

Four layers that make distributed streaming possible

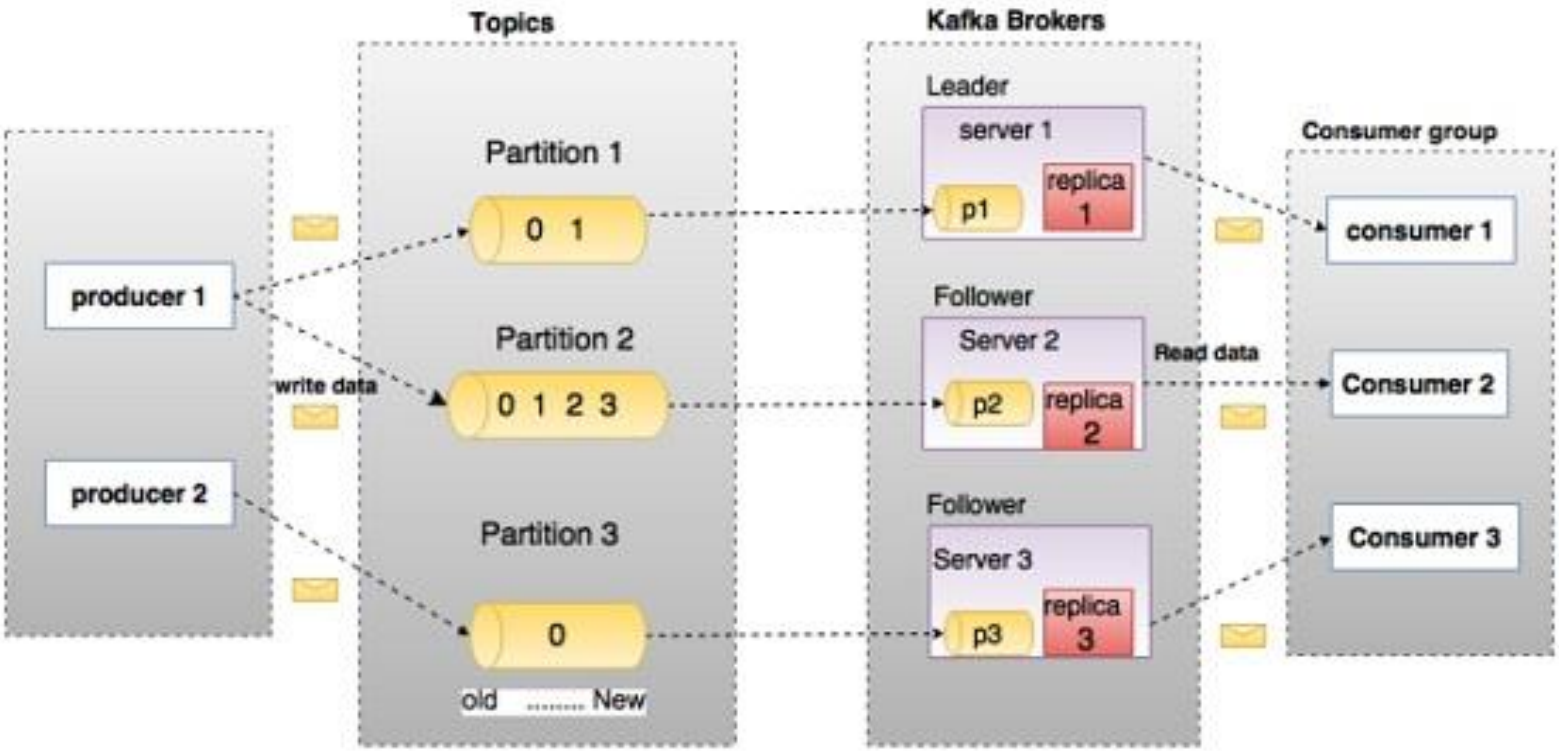
Broker Cluster

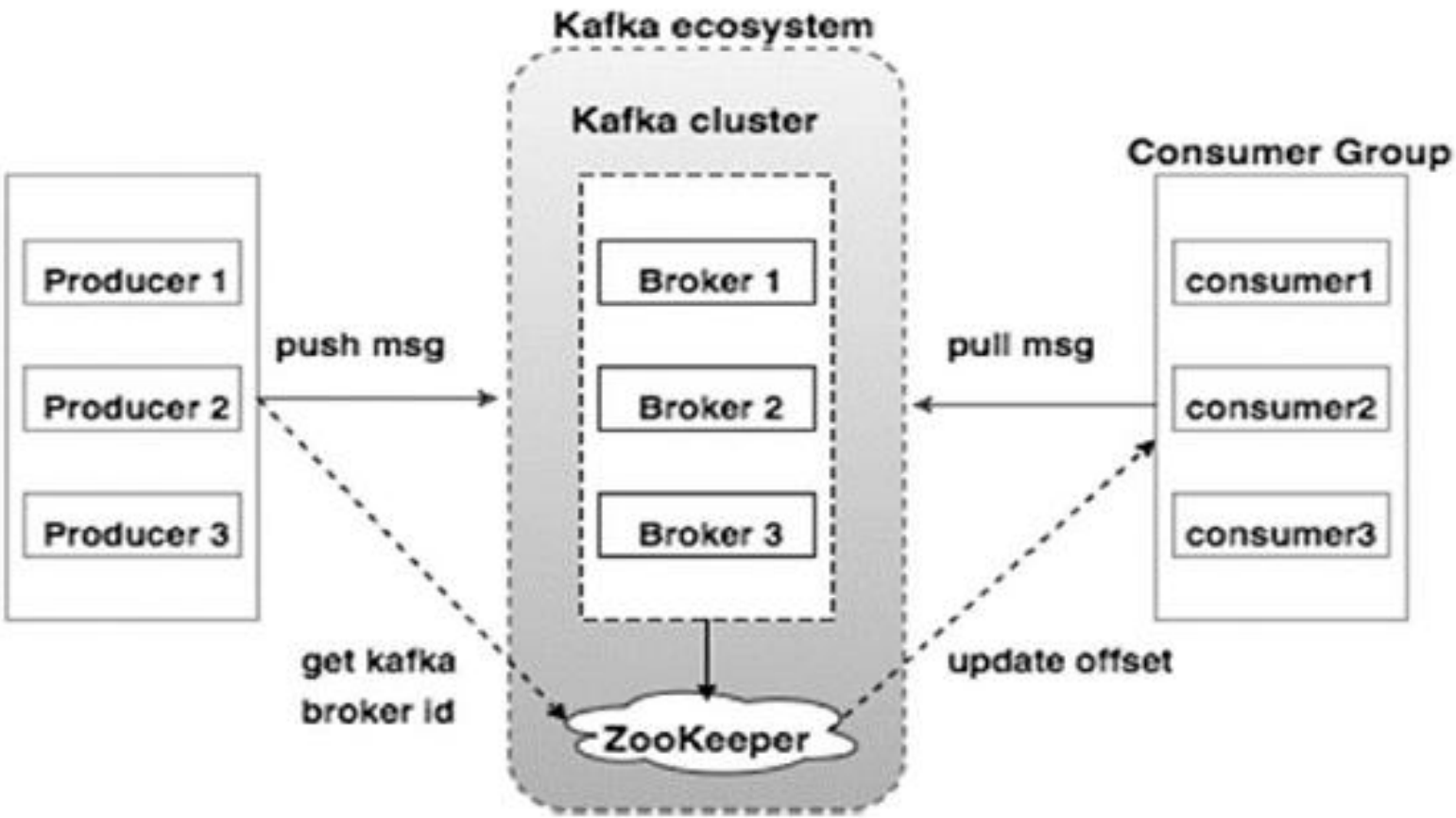
- Multiple broker nodes for load balance and fault tolerance
- Stateless by design — no state stored locally on the broker
- One broker instance can handle 100K+ reads/writes per second
- Leader election for partitions coordinated by ZooKeeper/KRaft

ZooKeeper (Legacy)

- Coordinates broker metadata and consumer offsets
- Notifies producers/consumers of broker arrivals and failures
- Manages leader election during broker failures
- Being deprecated in favor of KRaft in Confluent Platform 8+

Producers push data to brokers immediately — no wait for acknowledgement (configurable). Consumers pull asynchronously, track offsets independently, and can rewind or skip to any point in a partition.


Topics
Kafka Brokers
Leader
server 1
Consumer group
Partition 1
0 1
replica 1
p1
consumer 1
producer 1
Follower
Server 2
Partition 2
Read data
write data
0 1 2 3
p2
replica 2
Consumer 2
producer 2
Follower
Partition 3
Server 3
Consumer 3
replica 3
p3
0
old ..... New


Kafka ecosystem
Kafka cluster
Consumer Group
Producer 1
Producer 2
Producer 3
Broker 1
Broker 2
Broker 3
consumer1
consumer2
consumer3
push msg
pull msg
get kafka
broker id
update offset
ZooKeeper

03

KRaft Mode

Confluent Platform 8.0+ · ZooKeeper-Free · Raft-Based Metadata Consensus

Introduction to KRaft

Kafka Raft — the future of metadata management in Confluent Platform

ZooKeeper Mode (Legacy)

- Separate ZooKeeper cluster required
- Additional infra to deploy and manage
- Slower metadata propagation
- Cascading failure risk across systems
- Limited to ~200K partitions per cluster
- Leader failover takes seconds

KRaft Mode (Confluent 8.0+)

- Metadata management embedded in Kafka itself
- Eliminates external ZooKeeper dependency
- Faster metadata propagation across brokers
- Simplified deployment — fewer moving parts
- Supports up to 2M+ partitions per cluster
- Near-instantaneous controller failover

KRaft Architecture Deep Dive

How the Raft quorum manages Kafka metadata

Controller Quorum & Metadata Log

KRaft controllers form a Raft quorum that stores all topic, partition, ISR and config changes in a replicated metadata log — ensuring consistency across controllers.

Active Controller vs Followers

The active controller handles all broker RPCs. Follower controllers replicate metadata and serve as instant hot standbys for failover without re-election delay.

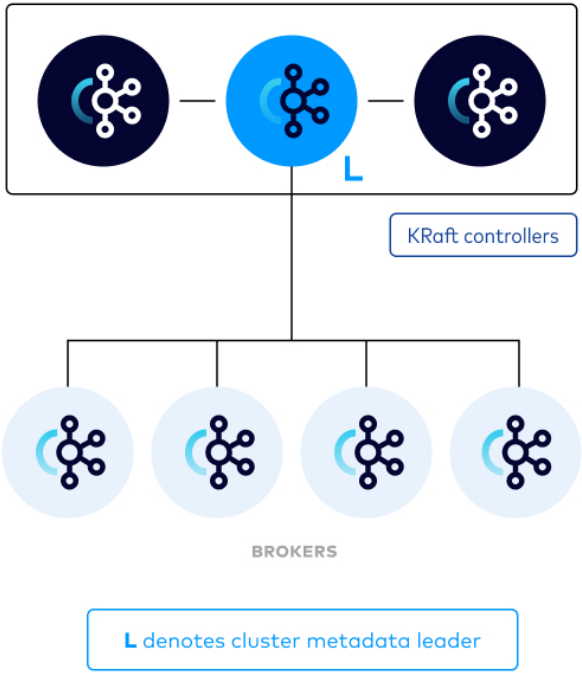
Snapshot & Recovery Mechanism

Controllers periodically write metadata snapshots to disk. On restart, a snapshot + delta replay eliminates full log replay — dramatically reducing recovery time.

Raft Consensus & Leader Election

Majority voting drives leader election and fault tolerance: 3-node quorum survives 1 failure; 5-node survives 2. No external coordinator required.

KRaft



L denotes cluster metadata leader

KRaft Scalability & Performance Gains

Partition limits, failover speed, and cluster-wide improvements

10×

more partitions
than ZooKeeper

2M+

partitions in a
single KRaft cluster

<1s

controller failover
(vs seconds on ZK)

3 or 5

nodes in quorum
for HA (majority voting)

Enhanced Metadata Propagation

KRaft's quorum controller minimises metadata propagation delay, enabling faster synchronisation across brokers and reducing cluster-wide downtime windows.

Operational Simplicity

One less cluster to provision, secure, monitor and upgrade. Simpler networking, fewer TLS certificates, and consolidated alerting for Kafka administrators.

High Availability Consensus

A 3-node quorum survives 1 controller failure; 5 nodes survive 2 simultaneous failures. Majority voting guarantees continued operation without external coordination.

KRaft Configuration Essentials

Core settings required for Confluent Platform production deployment

`process.roles`	`= controller | broker`	Specifies whether this node is a controller, broker, or both. Activates KRaft mode. Combined mode (both) is NOT supported in production.
`node.id`	`= unique integer per node`	Assigns a unique ID to each node. No two servers — controller or broker — may share the same node.id within a cluster.
`controller.quorum.voters`	`= {id}@{host}:{port}`	Defines all controllers participating in the Raft quorum. All nodes (brokers and controllers) must know the full quorum list.
`controller.listener.names`	`= CONTROLLER`	Lists controller listener names. Must match the listeners property and be consistent across the quorum voters list.
`metadata.log.dir`	`= /var/kafka/meta`	Stores the controller metadata log. Defaults to first entry in log.dirs if unset. Keep on fast, durable storage.
`inter.broker.listener.name`	`= REPLICATION`	Defines the listener used for inter-broker communication. Ensures secure replication between broker nodes.

04

Admin Tools & Best Practices

KRaft CLI Tools · Migration · Monitoring · Limitations

KRaft Administration Toolset

Command-line utilities for cluster insight, diagnostics and migration

`kafka-metadata-quorum`	**Inspect quorum state**	Describes the runtime status of the KRaft metadata partition. Connect via --bootstrap-server or --bootstrap-controller to inspect quorum leadership and follower lag.
`kafka-dump-log`	**Decode metadata records**	Decodes and prints metadata records from KRaft log segments and snapshots. Essential for deep inspection of stored partition and ISR state.
`kafka-metadata-shell`	**Interactive metadata explorer**	Interactive shell for exploring metadata partitions in real time. Supports SSL connections to running controllers for secure inspection.
`kafka-migration-check`	**Assess migration readiness**	Evaluates migration status from ZooKeeper to KRaft. Validates Confluent Platform transitions and highlights blockers before cutover.

Migration, Limitations & Best Practices

Transitioning safely from ZooKeeper and ensuring KRaft stability

⚠ Limitation

No Combined Mode in Production

Running a node as both broker AND controller is not supported in production environments due to security and operational gaps.

⚠ Limitation

Monitoring Tool Gaps

Certain tools, including Health+, may misreport KRaft controllers as brokers, impacting alert accuracy. Account for this during rollout.

✓ Best Practice

Use kafka-migration-check

Before migrating, run kafka-migration-check to validate readiness. Ensure each node's storage is formatted with a consistent cluster ID.

✓ Best Practice

Quorum Reconfiguration

Prior to Confluent Platform 7.9, controllers could not be added/removed dynamically. Plan quorum size carefully before initial deployment.

✓ Best Practice

Secure Metadata Communication

✓ Best Practice

Fast, Durable Metadata Storage

The Benefits of KRaft — Why It Matters

Simpler, faster, and more reliable Kafka metadata management

01

Unified Metadata

KRaft embeds metadata handling directly into Kafka — eliminating ZooKeeper and reducing operational surface area.

02

Improved Fault Tolerance

Raft-based consensus and majority voting ensure resilient controller failover with minimal downtime.

03

Operational Efficiency

Simplified configuration and deployment enable easier scaling and dramatically reduced administrative complexity.

04

Enhanced Scalability

Supports 10× more partitions than ZooKeeper-based clusters — enabling massive enterprise-scale deployments.

Key Takeaways

1	**Confluent = Kafka + Enterprise**	Schema Registry, ksqlDB, Connect, REST Proxy make Kafka production-ready at scale.
2	**Topics are durable logs**	Append-only, immutable, configurable retention. Not queues — logs with offsets.
3	**Partitioning = parallelism**	Key-based routing guarantees ordering. Hash function distributes evenly.
4	**KRaft eliminates ZooKeeper**	Confluent 8.0+ embeds metadata in Kafka itself for simpler, faster, bigger clusters.
5	**Admin tooling is first-class**	kafka-metadata-quorum, kafka-dump-log and the metadata shell give deep visibility.